

Sprache: Java 22+ (ohne Build-Tool, Multi-File Source Launcher) · Repository: github.com/phile2006/monte-Carlo-Simulation

1 – Ziel des Projekts

Es soll ein Kommandozeilen-Tool entwickelt werden, das die zukünftige Entwicklung eines Aktienkurses per **Monte-Carlo-Simulation** nachbildet: Statt einer einzelnen Prognose werden sehr viele mögliche Kursverläufe (Standard: 100.000 Pfade) zufällig erzeugt. Aus der Verteilung der Endpreise lassen sich dann Wahrscheinlichkeitsaussagen ableiten — etwa „Mit 5 % Wahrscheinlichkeit verliere ich über ein Jahr mehr als X %“ (Value at Risk) oder „Wie wahrscheinlich erreicht die Aktie den Zielkurs 120?“.

Das Projekt trainiert drei Dinge gleichzeitig: **Finanzmathematik** (stochastische Prozesse, Risikokennzahlen), **saubere Software-Architektur** (Modelle, Engine und Statistik strikt getrennt) und **parallele Programmierung** in Java (alle CPU-Kerne nutzen, trotzdem reproduzierbare Ergebnisse).

2 – Fachlicher Hintergrund

Geometrische Brownsche Bewegung (GBM)

Das Standardmodell für Aktienkurse. Der Kurs S wird in Tagesschritten $\Delta t = 1/252$ fortgeschrieben (252 Handelstage pro Jahr), mit der exakten Diskretisierung:

$$S_{t+\Delta t} = S_t \cdot \exp((\mu - \sigma^2/2) \cdot \Delta t + \sigma \cdot \sqrt{\Delta t} \cdot Z), \quad Z \sim N(0, 1)$$

Dabei ist μ die annualisierte Driftrate (erwartete Rendite) und σ die annualisierte Volatilität. Der Term $-\sigma^2/2$ ist die Itô-Korrektur: Ohne ihn wäre der Erwartungswert des simulierten Kurses falsch. Wichtige analytische Kontrollwerte: $E[S_T] = S_0 \cdot e^{\mu T}$ und $\text{Median}[S_T] = S_0 \cdot e^{(\mu - \sigma^2/2)T}$.

Merton-Jump-Diffusion

Die GBM unterschätzt extreme Ereignisse (Crashes). Das Merton-Modell ergänzt deshalb Sprünge: Zusätzlich zur normalen Diffusion tritt mit Rate λ (Sprünge pro Jahr, Poisson-verteilt) ein Sprung auf, dessen Log-Größe $J \sim N(\mu_J, \sigma_J^2)$ ist. Damit μ weiterhin die *gesamte* erwartete Rendite inklusive Sprünge bleibt, muss die Drift kompensiert werden:

$$\text{Drift-Kompensation: } \mu' = \mu - \lambda \cdot (E[e^J] - 1), \quad E[e^J] = \exp(\mu_J + \sigma_J^2/2)$$

Risikokennzahlen

- **Value at Risk (VaR):** die Rendite am 5. bzw. 1. Perzentil der Ergebnisverteilung — „schlechter als das wird es nur in 5 % / 1 % der Fälle“.
- **Conditional VaR (CVaR / Expected Shortfall):** der Mittelwert der Renditen *innerhalb* dieses Tails — „wenn es schlecht läuft, wie schlecht läuft es dann im Schnitt?“.
- **Maximum Drawdown:** der größte prozentuale Rückgang vom bisherigen Höchststand innerhalb eines Pfades — wird pro Pfad berechnet und als Verteilung berichtet.

3 – Aufgabenstellung

Teilaufgabe 1 — Modelle

- Definiere ein Interface `PriceModel` mit einer Methode, die aus dem aktuellen Kurs und einer Zufallsquelle den nächsten Tageskurs erzeugt.
- Implementiere `GbmModel` (exakte Diskretisierung aus Abschnitt 2) und `JumpDiffusionModel` (GBM-Schritt + Poisson-Sprünge mit Drift-Kompensation aus 2.2).

Teilaufgabe 2 — Parallele Simulations-Engine

- Simuliere N Pfade über T Handelstage; pro Pfad werden nur **Endpreis** und **maximaler Drawdown** gespeichert (nicht der ganze Pfad — Speicher!).
- Verteile die Pfade über alle CPU-Kerne (z. B. parallele Streams oder `ExecutorService`).
- **Reproduzierbarkeit:** Jeder Pfad bekommt seinen eigenen Zufallsstrom aus einem splittbaren Generator (`SplittableRandom` / `RandomGenerator.SplittableGenerator`), abgeleitet aus einem festen Seed. Gleicher Seed $\Rightarrow$ identisches Ergebnis, unabhängig von der Thread-Anzahl.

Teilaufgabe 3 — Statistik & Report

- Berechne aus den Endpreisen: Mittelwert, Median, Standardabweichung, Perzentile (1, 5, 25, 50, 75, 95, 99).
- Berechne Horizont-Renditen, VaR (5 %/1 %) und CVaR als Tail-Mittelwert.
- Gib ein **ASCII-Histogramm** der Endpreisverteilung in der Konsole aus.
- Optional pro Aufruf: $P(S_T \geq \text{Zielkurs})$ bei `--target`, CSV-Export von 20 Beispielpfaden bei `--export`.

Teilaufgabe 4 — CLI

Alle Parameter als Kommandozeilen-Flags mit sinnvollen Defaults:

Flag	Default	Bedeutung
<code>--s0</code>	100	Startkurs
<code>--mu</code>	0.08	annualisierte Drift (dezimal)
<code>--sigma</code>	0.20	annualisierte Volatilität (dezimal)
<code>--days</code>	252	Horizont in Handelstagen
<code>--paths</code>	100000	Anzahl simulierter Pfade
<code>--seed</code>	42	RNG-Seed (reproduzierbar pro Seed)
<code>--model</code>	gbm	gbm oder jump
<code>--lambda</code>	4	(jump) erwartete Sprünge pro Jahr
<code>--jump-mean</code>	-0.03	(jump) mittlere Log-Sprunggröße

Flag	Default	Bedeutung
--jump-std	0.05	(<i>jump</i>) Streuung der Log-Sprunggröße
--target	—	zusätzlich $P(\text{Endpreis} \geq \text{target})$ ausgeben
--export	—	CSV-Datei mit 20 Beispielpfaden

4 – Technische Anforderungen

- Java 22+, **kein Build-Tool**: Start direkt per `java MonteCarloSim.java` (Multi-File Source Launcher).
- Strikte Trennung der Verantwortlichkeiten auf Dateiebene: `MonteCarloSim.java` (CLI + Report), `Simulator.java` (parallele Engine), `PriceModel.java/GbmModel.java/JumpDiffusionModel.java` (Modelle), `Stats.java` (Perzentile, VaR/CVaR, Histogramm).
- Ungültige Eingaben (z. B. $\sigma < 0$, $\text{paths} \leq 0$, unbekanntes Modell) führen zu einer klaren Fehlermeldung und Abbruch — keine stillen Korrekturen.

5 – Abnahmekriterien & Verifikation

Das Projekt gilt als fertig, wenn:

- **Analytischer Sanity-Check**: Mit den Defaults liegen simulierter Mittelwert und Median der Endpreise innerhalb des Monte-Carlo-Fehlers an den analytischen Werten $S_0 \cdot e^{\mu T} \approx 108.33$ und $S_0 \cdot e^{(\mu - \sigma^2/2)T} \approx 106.18$.
- **Reproduzierbarkeit**: Zwei Läufe mit gleichem Seed liefern bitgleiche Kennzahlen — auch bei unterschiedlicher Thread-Anzahl.
- **Jump-Kompensation**: Der Mittelwert der Endpreise im Jump-Modell entspricht ebenfalls $S_0 \cdot e^{\mu T}$ (weil die Drift kompensiert wird).
- **Plausibilität**: Höheres σ verbreitert die Verteilung (VaR wird schlechter); negative Sprünge machen den linken Tail dicker (CVaR deutlich unter VaR).
- 100.000 Pfade $\times$ 252 Tage laufen in wenigen Sekunden.

6 – Mögliche Erweiterungen

- Heston-Modell (stochastische Volatilität) als drittes `PriceModel`.
- Antithetic Variates zur Varianzreduktion (halbiert den Monte-Carlo-Fehler fast gratis).
- Bewertung europäischer Optionen: Payoff am Laufzeitende diskontieren und mit Black-Scholes vergleichen.